WNDRVR
SUPPORT NETWORK

VXWORKS 7 RISC-V ARCHITECTURE GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 RISC-V Architecture Guide, SR0650

26 June 2020

1. OVERVIEW

1. Supported RISC-V Cores

RISC-V (pronounced "risk-five") is an open standard instruction set architecture (ISA), based on established reduced instruction set computer (RISC) principles.

The RISC-V ISA is free and distributed with a permissive open source license. The RISC-V ISA is defined as a base integer ISA (which must be present in any implementation), plus optional extensions to the base ISA.

Table 1. Supported RISC-V Cores

CPU Type	Description	32-bit Support?	64-bit Support?	RISC-V arch version
RISCV	Includes IMAFDC standard extensions support. For details about the various VSB options for RISC-V extensions see Standard Extensions Support on page 5	Yes	Yes	Privileged Architecture Version 1.10 User Level ISA Version 2.0

2. RISC-V

1. Interface Variations

These are particular features and functions that are specific to RISC-V targets.

The unique characteristics are available only on RISC-V targets, use parameters specific to RISC-V targets, or have special restrictions or characteristics on RISC-V targets.

cacheLib on page 2cacheTextUpdate( ) is the only function supported.
dbgLib on page 2RISC-V architecture supports both software and hardware breakpoint.
intALib on page 2
vxALib on page 3
vxMemProbeArchALib on page 3

1.1. cacheLib

cacheTextUpdate( ) is the only function supported.

All the other cache functions always return **ERROR**. RISC-V architecture does not provide any mechanism to turn the cache on or off, so the instruction and data cache are always enabled and cannot be disabled.

As of now, RISC-V architecture does not define any instruction to invalidate or flush data caches, hence a full coherency model is guaranteed by hardware design between processor cores and any DMA masters.

Parent topic: Interface Variations on page 2

1.2. dbgLib

RISC-V architecture supports both software and hardware breakpoint.

When you set a software breakpoint, VxWorks replaces the instruction with an ebreak instruction and it restores the original code when the breakpoint is removed. If memory is examined or disassembled, the original code is shown. The instruction cache is purged each time VxWorks changes an instruction to a software break instruction.

Hardware breakpoint is not supported by VxWorks as VxWorks runs in supervisor mode and has no privilege to access the debug registers to manipulate hardware breakpoints.

Parent topic: Interface Variations on page 2

1.3. intALib

intLock( ) and intUnlock( )

> The intLock( ) function returns the SIE bit from the SSTATUS as the lock-out key for the interrupt level prior to the call to intLock( ). The intUnlock( ) function takes this value as a parameter. For RISC-V processors, these functions control the CPU interrupt mask directly. They do not manipulate the interrupt levels in the interrupt controller chip.
>
> The VxWorks SMP kernel does not allow interrupt locking on all the CPUs in the system and so these functions are not available in VxWorks SMP.

intCpuLock( ) and intCpuUnlock( )

These functions are called by a VxWorks task or interrupt service routine, and control whether interrupts are enabled or disabled on that context's local CPU. The intCpuLock( ) function returns the SIE bit from the SSTATUS on the local CPU as the lock-out key for the interrupt level prior to the call to intCpuLock( ). The intCpuUnlock( ) function takes this value as a parameter. For RISC-V processors, these functions control the local CPU interrupt mask directly. They do not manipulate the interrupt levels in the interrupt controller chip.

The VxWorks SMP kernel allows interrupt locking on individual local CPUs in the system, and so these functions are available in VxWorks SMP. On the uniprocessor VxWorks, these functions are equivalent to intLock( ) and intUnlock( ).

Note: On VxWorks SMP, the interrupts on the other CPUs in the system are not affected by these functions. Consequently, they are not suitable as a mutual exclusion mechanism unless all the tasks and/or interrupt service routines participating in the mutual exclusion have a single CPU affinity to the same CPU.

Parent topic: Interface Variations on page 2

1.4. vxALib

vxTas( )

The test-and-set primitive vxTas( ) provides a C-callable interface to the RISC-V LR/SC (load/store exclusive) instructions.

Parent topic: Interface Variations on page 2

1.5. vxMemProbeArchALib

RISC-V architecture supports the vxMemProbe( ) function, which probes an address for a bus error, by trapping both load page fault and store page fault.

Parent topic: Interface Variations on page 2

2. Architecture Considerations

There are characteristics of RISC-V processors that you should keep in mind as you write a VxWorks application. For comprehensive documentation on the RISC-V architectures and specific processors, see the *RISC-V Instruction Set Manual Volume I & II* and the data sheets for your target processor.

Processor Mode on page 4
Supervisor Binary Interface on page 4RISC-V operating systems interface with a supervisor execution environment (SEE) via a supervisor binary interface (SBI).
Byte Order on page 4RISC-V architecture only supports little-endian byte order.
Code Models on page 4RISC-V supports two code models (**medlow** and **medany**) as of now.
Standard Extensions Support on page 5RISC-V optional extensions can be optionally turned on by various VSB options.
Floating Point Support on page 6 VxWorks kernel, DKM and RTP can be compiled with hardware floating-point support enabled or disabled.
Hardware Breakpoints on page 6 Hardware breakpoints are not supported by VxWorks.
Unaligned Accesses on page 6RISC-V architecture does not support unaligned data access.
Cache on page 7
Interrupts and Exceptions on page 7RISC-V supports two interrupt trap modes called direct mode and vectored mode.
Memory Management on page 7A RISC-V processor that runs in supervisor mode (S-mode) provides memory management functionalities like address translation and protection.
Physical Addressing Support on page 8 VxWorks for RISC-V supports 64-bit physical addressing size (**PHYS_ADDR**).

Expose the Target Memory Map Programmatically on page 8
Memory Layout on page 9 VxWorks memory layout (real or virtual, as appropriate) is the same for all RISC-V processors.
64-bit DKM and Branch Island on page 10
Thread Local Storage on page 10

2.1. Processor Mode

At any time, a RISC-V hardware thread (hart) is running at some privilege level encoded as a mode in one or more CSRs (control and status registers). Three RISC-V privilege levels are currently defined as shown in the RISC-V privilege levels table below.

Table 1. RISC-V Privilege Level

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

Machine mode (M-mode) can be used to manage secure execution environments on RISC-V. User mode (U-mode) and supervisor mode (S-mode) are intended for conventional application and operating system usage respectively. VxWorks kernel runs on S-mode and RTPs run on U-mode.

Parent topic: Architecture Considerations on page 3

2.2. Supervisor Binary Interface

RISC-V operating systems interface with a supervisor execution environment (SEE) via a supervisor binary interface (SBI).

SBI comprises the user-level and supervisor-level ISA together with a set of SBI function calls. Using a single SBI across all SEE implementations allows a single operating system binary image to run on any SEE. The SEE can be a simple boot loader and BIOS-style IO system in a low-end hardware platform, or a hypervisor provided virtual machine in a high-end server, or a thin translation layer over a host operating system in an architecture simulation environment.

VxWorks requires an SEE that provides RISC-V SBI specification version 0.2 defined interfaces. OpenSBI is the only SBI implementation that is validated to work with VxWorks. OpenSBI version 0.7 release or above is required to work with VxWorks.

Parent topic: Architecture Considerations on page 3

2.3. Byte Order

RISC-V architecture only supports little-endian byte order.

Parent topic: Architecture Considerations on page 3

2.4. Code Models

RISC-V supports two code models (**medlow** and **medany**) as of now.

For **medlow**, the compiler generates code for the medium-low code model. The program and it's statically defined symbols must lie within a single 2 GiB address range and must lie between absolute addresses -2 GiB and +2 GiB. Programs can be statically or dynamically linked.

For **medany**, the compiler generates code for the medium-any code model. The program and it's statically defined symbols must be within any single 2 GiB address range. Programs can be statically or dynamically linked.

The major difference of the two code models are instruction pairs compiler generates for data access. For **medlow** it is "lui/ld" pairs while for **medany** it is "auipc/ld" pairs. It is natural to select **medlow** code model for 32-bit in both kernel and user space. For 64-bit **medlow** can be supported with the restriction of kernel system memory region start address (**KERNEL_SYS_MEM_RGN_BASE**) higher than 0xffffffff80000000, as under **medlow** code model, the "lui" instruction will automatically do sign-extension in 64-bit. With the above restriction we can select **medlow** for 64-bit in both kernel and user space too.

VxWorks uses **medlow** code model by default for kernel / DKM / RTP build. To change the code model to **medany**, reconfigure the VSB by selecting the option **_WRS_CONFIG_CMODEL_MEDANY**.

Parent topic: Architecture Considerations on page 3

2.5. Standard Extensions Support

RISC-V optional extensions can be optionally turned on by various VSB options.

Table 1. RISC-V Optional Extensions

Standard Extension	VSB Option	Description
M standard extension	_WRS_CONFIG_ISA_M	Enable this option if the target supports integer multiplication and division instruction extension.
A standard extension	_WRS_CONFIG_ISA_A	Enable this option if the target supports the atomic instruction extension. This option is mandatory in SMP mode.
F standard extension	_WRS_CONFIG_ISA_F	Enable this option if the target supports the single-precision floating point instruction extension.
D standard extension	_WRS_CONFIG_ISA_D	Enable this option if the target supports the double-precision floating point instruction extension. Note, this means the F standard extension for single-precision floating point is also supported.
C standard extension	_WRS_CONFIG_ISA_C	Enable this option if the target supports the compressed instruction extension.

A specific BSP for a RISC-V processor has corresponding ISA standard extension VSB options selected by default. Below is an example of BSP configuration:

```
config BSP_CONFIG
    bool
    depends on RISCV
```


WNDRVR

```
select ISA_M
select ISA_A
select ISA_F
select ISA_D
select ISA_C
default y
```

Parent topic: Architecture Considerations on page 3

2.6. Floating Point Support

VxWorks kernel, DKM and RTP can be compiled with hardware floating-point support enabled or disabled.

Three floating point modes are supported: **khard**, **ksoft** and **soft**.

The availability of the **khard**, **ksoft**, and **soft** floating point modes allow a system designer to choose the appropriate mode based on performance requirements.

- The **khard** FP mode should be selected to maximize throughput performance in both the kernel and user-space but at the expense of increasing task switch latency for all tasks and ISR contexts. When the **khard** FP mode is selected, the VX_FP_TASK option bit is implicitly set by the operating system, for both kernel and RTP tasks, and implicit fppSave( )/fppRestore( ) operations are performed when invoking an ISR.
- The **ksoft** FP mode should be selected to maximize throughput performance in user-space and to maximize real-time performance in the kernel with respect to kernel task context switch latency and ISR latency. When the **ksoft** FP mode is selected, the VX_FP_TASK option bit is implicitly set by the operating system only for RTP tasks, and implicit fppSave( )/fppRestore( ) operations are not performed when invoking an ISR.
- The **soft** FP mode should be selected to maximize real-time performance in both the kernel and user-space with regards to task switch latency for both kernel and RTP tasks, and ISR latency. When the **soft** FP mode is selected, the VX_FP_TASK option bit is not implicitly set by the operating system for any task and implicit fppSave( )/fppRestore( ) operations are not performed when invoking an ISR.

The VSB is configured by default to use hardware floating point, aka **khard** FP mode. This configuration is the best choice for most users. It uses the floating point coprocessor to do floating-point computing, rather than performing software simulated floating point calculation.

When VxWorks is configured to use the software floating point, it uses the software FP calling convention (ABI). The software FP ABI uses the general purpose registers rather than the floating point coprocessor to pass floating point parameters to functions, and return the floating point result to the caller. The extra cost for the software ABI is the movement and conversion between general purpose registers and floating point coprocessor registers.

Parent topic: Architecture Considerations on page 3

2.7. Hardware Breakpoints

Hardware breakpoints are not supported by VxWorks.

This is because VxWorks runs in supervisor mode and has no privilege to access the debug registers to manipulate hardware breakpoints.

Parent topic: Architecture Considerations on page 3

2.8. Unaligned Accesses

RISC-V architecture does not support unaligned data access.

Unaligned loads and stores are trapped and emulated by M-mode SBI firmware which is transparent to VxWorks. VxWorks does not enable strict alignment access.

Parent topic: Architecture Considerations on page 3

2.9. Cache

RISC-V architecture does not provide any mechanism to explicitly enable or disable cache. Instruction cache and data cache are always on.

Parent topic: Architecture Considerations on page 3

2.10. Interrupts and Exceptions

RISC-V supports two interrupt trap modes called direct mode and vectored mode.

Currently only the direct mode is supported in VxWorks.

Parent topic: Architecture Considerations on page 3

2.11. Memory Management

A RISC-V processor that runs in supervisor mode (S-mode) provides memory management functionalities like address translation and protection.

Virtual Addressing Mode on page 7
Variable Page Sizes on page 7 By default, addresses are mapped in blocks of 4 KB, also called small pages.
Address Space Identifiers on page 8

Parent topic: Architecture Considerations on page 3

Virtual Addressing Mode

32-bit RISC-V processor supports SV32 virtual addressing mode.

64-bit RISC-V processor supports SV39 virtual addressing and SV48 virtual addressing mode.

The mapping of SV48 mode is based on four level page table. However SV39 mode translates virtual address to physical address via a three level page table. By default VxWorks uses SV39 mode for 64-bit RISC-V.

To change the virtual addressing mode in 64-bit mode, reconfigure the VSB by selecting the _WRS_CONFIG_SATP_MODE_SV48 option.

Parent topic: Memory Management on page 7

Variable Page Sizes

By default, addresses are mapped in blocks of 4 KB, also called small pages.

The properties of these pages are stored in MMU translation tables. It is also possible to map addresses in 4 MiB sized mega pages for RV32, or 2 MiB sized mega pages and 1 GiB sized giga pages for RV64 SV39 virtual addressing.

Support for larger page sizes allows virtual memory management to be more efficient with fewer translation table walks or translation lookaside buffer (TLB) misses. Dynamic and static methods for mapping larger page sizes provide mechanisms for increasing system performance.

Parent topic: Memory Management on page 7

Address Space Identifiers

RISC-V architecture supports 512 unique address space identifiers (ASIDs) for RV32 and 65536 ASIDs for RV64, which are used to improve context switch performance with real-time processes (RTPs) by reducing the need to flush and invalidate TLBs.

Among all these ASIDs, one ASID is reserved for kernel use. 0 is not a special address space identifier, so kernel employs ASID 0. The first created RTP will use ASID 1 and ASID plus one for every subsequent start up RTP. That means kernel tasks share the same MMU map with ASID 0 and every RTP has its own MMU map with its special ASID.

ASID is an optional feature of the architecture and is transparent to the operating system. VxWorks supports running on hardware with or without ASID support without any special VSB configuration.

For hardware that does not support ASID, the ASID encoded in the instructions will be safely ignored by the RISC-V processor, as if kernel and all RTPs are sharing the same ASID 0.

Parent topic: Memory Management on page 7

2.12. Physical Addressing Support

VxWorks for RISC-V supports 64-bit physical addressing size (**PHYS_ADDR**).

On RISC-V 32-bit, VxWorks supports 34 **PHYS_ADDR** bits.

On RISC-V 64-bit, VxWorks supports 56 **PHYS_ADDR** bits.

Parent topic: Architecture Considerations on page 3

2.13. Expose the Target Memory Map Programmatically

The BSP statically defines the memory regions that are available to VxWorks while it is running on the target. For example, on the **sifive_generic** BSP, the memory map for the **sifive_generic** board is defined in the file **hifive-unleashed-a00.dts** as follows:.

```
memory@80000000 {
                device_type = "memory";
                reg = <0x0 0x80000000 0x2 0x00000000>;
        };
```

The memory region occupied by the SBI firmware is protected and the information is passed via the device tree under the /reserved-memory node. VxWorks parses the /reserved-memory node to exclude that memory region for the VxWorks application, BSP and driver code. VxWorks makes a call to the function sysMemDescGet( ) when initializing the address space manager, etc to retrieve the memory map configuration.

For example:

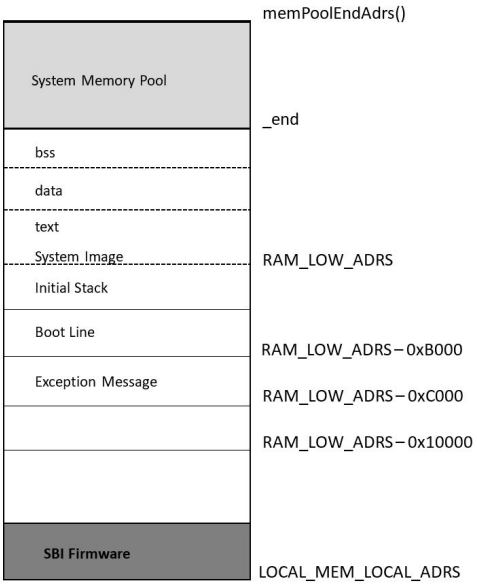
```
LOCAL PHYS_MEM_DESC sysPhysRamDesc[32];
LOCAL STATUS success;

success = sysMemDescGet(MEM_DESC_RAM, 0, sysPhysRamDesc);
```

Parent topic: Architecture Considerations on page 3

2.14. Memory Layout

VxWorks memory layout (real or virtual, as appropriate) is the same for all RISC-V processors.

The following figure shows the memory layout:



System Memory Pool

All available system memory is managed using the global RAM pool, and the adrSpaceShow( ) function can be used to get information about the usage.

System Image

VxWorks (has three sections: text, data, and bss). The entry point for VxWorks is at the start of this region. This is configured within the BSP.

Initial Stack

Initial stack for usrInit( ), until usrRoot( ) is allocated a stack. This is configured within the BSP.

Boot Line

ASCII string of boot parameters.

Exception Message

ASCII string of fatal exception message.

DTB Relocation

Copied DTB data.

SBI Firmware

Memory occupied by SBI firmware. VxWorks does not use this memory region and marks it as reserved.

Parent topic: Architecture Considerations on page 3

2.15. 64-bit DKM and Branch Island

By default, the 64-bit version of DKM is loaded at kernel proximity heap which is located no further than 2GB from the kernel code. In this case, DKM's code model can be set to either **medlow** or **medany**, because for all relocation types generated in both code models, the symbol addresses are within the limit range (+/-2GB from 0 or +/-2GB from PC) and can be fit into the bits specified by the relocation.

The DKM can also be loaded at kernel common heap. It is located further than 2GB from the kernel code in 64-bit system. This will lead to relocation overflows. A **.branchisland** section helps resolve this issue. A jump instruction in DKM to a symbol in the kernel can be modified to jump to an address within **.branchisland** section, that address will hold the instructions that jump to the symbol in the kernel. In this case, the DKM must be built with **medany** code model, but kernel image itself can still be built with either **medlow** or **medany** code model.

> Note: Access to global data in the kernel from 64-bit DKM located at kernel common heap is not supported.

Parent topic: Architecture Considerations on page 3

2.16. Thread Local Storage

Local exec model is a completely static thread local storage (TLS) model where there is only one module – the statically linked executable. All the TLS variables are known at link time. This model is suitable for VxWorks kernel.

Initial exec model can reference TLS variables which are available as part of the initial static TLS template. This template is composed of all TLS blocks that are available at process startup, plus a small backup reservation. In this model, the thread pointer-relative offset for a given variable x is stored in the GOT (Global Offsets Table) entry for x. This model is used for VxWorks RTP executables both statically and dynamically linked.

Global dynamic model is the most generic model. Code compiled with it can be used everywhere and it can access variables defined anywhere else. This model is used for shared libraries.

Parent topic: Architecture Considerations on page 3

3. Reference Material

The information given in this section is current at the time of writing. If you decide to use these documents, you may want to contact the manufacturer for the most current version.

RISC-V Development Reference Documents

- RISC-V Instruction Set Manual Volume I: User-Level ISA, Version 2.2

WNDRVR

- RISC-V Instruction Set Manual Volume II: Privileged Architecture, Version 1.10
- RISC-V Assembly Programmer's Manual
- RISC-V ELF psABI specification
- RISC-V Supervisor Binary Interface Specification